

Exercise 2:

Big O notation is a way to describe time and space complexity of an algorithm based on the input.

As input size increases we can see how it affects the run time. This allows us to measure and compare different algorithms.

For Linear Search:

- Best case: $O(1)$
- Average case: $O(n)$
- Worst case: $O(n)$

For Binary Search:

- Best case: $O(1)$
- Average case: $O(\log n)$
- Worst case: $O(\log n)$

Binary Search is better especially as input size becomes very large.

Code:

```
1  class Product{
2      int productId;
3      String productName;
4      String category;
5
6      Product(int productId, String productName, String category){
7          this.productId = productId;
8          this.productName = productName;
9          this.category = category;
10     }
11
12     @Override
13     public String toString() {
14         return productId + " - " + productName + " - " + category;
15     }
16 }
```

```

18 public class Main {
19     static int linearSearch(Product[] arr, int n, int key){
20         for(int i=0; i<n; i++){
21             if (arr[i].productId == key)
22                 return i;
23         }
24         return -1;
25     }
26
27     static int binarySearch(Product[] arr, int n, int key){
28         int low = 0, high = n, mid;
29
30         while(low<=high){
31             mid = low + (high-low)/2;
32
33             if(arr[mid].productId == key)
34                 return mid;
35
36             else if(arr[mid].productId < key)
37                 low = mid+1;
38
39             else
40                 high = mid-1;
41         }
42         return -1;
43     }

```

```

44 public static void main(String[] args) {
45     int n = 5;
46     Product[] prods = new Product[n];
47     prods[0] = new Product(productId: 1, productName: "Chair", category: "Furniture");
48     prods[1] = new Product(productId: 2, productName: "Table", category: "Furniture");
49     prods[2] = new Product(productId: 3, productName: "Phone", category: "Electronics");
50     prods[3] = new Product(productId: 4, productName: "Notebook", category: "Stationary");
51     prods[4] = new Product(productId: 5, productName: "Pen", category: "Stationery");
52
53     int searchId = 4;
54
55     System.out.println(x: "Linear Search:");
56     int idx = linearSearch(prods, n, searchId);
57     if(idx != -1)
58         System.out.println("Product found: " + prods[idx]);
59     else
60         System.out.println(x: "Product not found");
61
62     System.out.println(x: "Linear Search:");
63     idx = binarySearch(prods, n, searchId);
64     if(idx != -1)
65         System.out.println("Product: " + prods[idx]);
66     else
67         System.out.println(x: "Product not found");
68 }
69 }

```

Output:

```
Linear Search:  
Product found: 4 - Notebook - Stationary  
Linear Search:  
Product: 4 - Notebook - Stationary
```

Binary Search is more suitable as it has time complexity of $O(\log n)$ unlike Linear Search with $O(n)$. As e-commerce platform will need to look through hundreds of thousands of products, using binary search increases search speed.